INTELLIDOCS: AN INTELLIGENT DOCUMENT Q&A AND RETRIEVAL SYSTEM

A report submitted in partial fulfillment of the requirements for the Award of Degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

by
Kotra Rupak Srinivas
Y23ACS485

Department of Computer Science and Engineering

BAPATLA ENGINEERING COLLEGE
(Autonomous)
Affiliated to Acharya Nagarjuna University, Guntur
Bapatla - 522102, Andhra Pradesh

2026

BAPATLA ENGINEERING COLLEGE (AUTONOMOUS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Internship report entitled **"INTELLIDOCS: AN INTELLIGENT DOCUMENT Q&A AND RETRIEVAL SYSTEM"** being submitted by **Kotra Rupak Srinivas** bearing Reg No: **Y23ACS485** is a record of bonafide work done by him and submitted during the academic year **2026–2027** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING**, after successfully completing his short-term internship program titled **"Generative AI with LLMs"** at **SkillDzire Technologies Private Limited, Hyderabad** from **25-05-2026 to 06-07-2026**.

Department Internship Coordinator **Head of the Department**

DECLARATION

I hereby declare that the dissertation/internship report entitled **"INTELLIDOCS: AN INTELLIGENT DOCUMENT Q&A AND RETRIEVAL SYSTEM"** submitted for the B.Tech Degree is my work, completed under the guidance of my company mentor at **SkillDzire Technologies Private Limited, Hyderabad**. This dissertation has not formed the basis for the award of any degree, associates, fellowship, or other similar academic titles in this or any other university.

Place: Bapatla
Date: 27-07-2026

Kotra Rupak Srinivas
Y23ACS485

INTERNSHIP CERTIFICATE ISSUED BY COMPANY

Figure: Certificate of Internship from SkillDzire

COMPANY PROFILE AND EXTERNAL GUIDE DETAILS

1. Company Profile: SkillDzire Technologies Private Limited

SkillDzire is an innovative educational technology platform based in Hyderabad, India, dedicated to bridging the gap between industry requirements and academic learning. The organization specializes in training students and professionals in emerging technological domains, including Artificial Intelligence, Machine Learning, Generative AI, Cloud Computing, and Web Development. SkillDzire is recognized and certified by major technological bodies, including the AICTE (All India Council for Technical Education) and the APSCHE (Andhra Pradesh State Council of Higher Education), allowing them to deliver certified practical programs. The company's primary focus is project-based hands-on learning, where students are guided by experienced industrial mentors to solve real-world problems and build industry-grade applications.

During this internship program, the workspace, resources, and instructional materials were focused on 'Generative AI with LLMs'. This domain includes text vector embeddings, Retrieval-Augmented Generation (RAG) concepts, prompt tuning, semantic vectors search, and integration of LLM APIs (such as Google Gemini, OpenAI, and LangChain orchestration) into custom workflows.

2. External Guide Details

The internship program was organized under the guidance of industry experts from SkillDzire. The external guide and authorized signatory for the internship credentials is:

- Name: M. Srikanth
- Designation: Technical Lead and Authorized Signatory
- Organization: SkillDzire Technologies Private Limited, Hyderabad, India
- Domain: Generative AI and Cloud Architectures
- Email / Contact: support@skilldzire.com

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my internship host organization, SkillDzire Technologies Private Limited, Hyderabad, for providing me with the opportunity to carry out my short-term internship program on 'Generative AI with LLMs'. I am highly indebted to the team of mentors at SkillDzire, particularly M. Srikanth, for their valuable lectures, instruction, and project specifications which guided the development of the IntelliDocs system.

I express my sincere thanks and deep gratitude to our Principal, Bapatla Engineering College, Bapatla, for providing excellent infrastructural support and encouragement. I am also extremely grateful to the Head of the Department of Computer Science and Engineering for his constant support and guidance throughout my academic tenure.

I would also like to thank the Department Internship Coordinator for coordinating and verifying the internship documents, evaluations, and guidelines. Their timely advice and regular progress updates were instrumental in compiling this final report.

Finally, I express my warm gratitude to my family and peers who have supported me in this learning curve and provided critical feedback on the user interface and overall experience of the IntelliDocs application.

Kotra Rupak Srinivas
Y23ACS485
BEC, Bapatla

ABSTRACT

Large Language Models (LLMs) have revolutionized natural language understanding, but they suffer from severe context limitations, knowledge cut-offs, and tendencies to hallucinate when asked about specific, proprietary, or locally stored documents. Retrieval-Augmented Generation (RAG) addresses these issues by combining semantic document search with LLM response generation. This report details the design and implementation of IntelliDocs, a completely on-device, zero-database-cost RAG system built to parse, index, and search local files using natural language.

IntelliDocs features an offline document parsing pipeline and a real-time online Q&A query loop. In the ingestion phase, the pipeline dynamically loads files from a local directory supporting multiple formats—PDFs, Word documents, PowerPoint presentations, tabular CSV files, markdown, and plain text. The extracted texts are segmented using recursive overlap-protected chunking algorithms to preserve context. High-dimensional semantic embeddings are computed for each text chunk using Google's generative AI models and indexed in a localized FAISS (Facebook AI Similarity Search) database stored directly on disk. In the Q&A loop, incoming user queries are mapped to the same vector space, enabling cosine-similarity comparisons against the local FAISS index. The top matching chunks are extracted, concatenated with a sliding conversation history buffer of the last three exchanges to resolve pronouns, and passed to a context-grounded prompt executed against the fast and cost-effective gemini-3.6-flash LLM. System testing confirms precise document-grounded answers with zero server database overhead and high data privacy.

Keywords: Generative AI, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), FAISS, Semantic Vector Embeddings, LangChain, Google Gemini API.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Introduction to Generative AI & RAG

Generative Artificial Intelligence represents a paradigm shift in computing, allowing algorithms to generate novel content (including text, code, images, and structured databases) based on learned patterns from massive datasets. At the heart of text-based Generative AI are Large Language Models (LLMs), neural networks that utilize the Transformer architecture to predict subsequent tokens and construct coherent, contextually appropriate text streams. While LLMs demonstrate impressive writing and reasoning capabilities, they are statically bound to the information present in their pre-training weights. Consequently, they cannot retrieve details about documents generated after their knowledge cutoff date, and they lack access to private, proprietary local databases. When forced to reply to questions beyond their training boundaries, LLMs frequently fabricate responses, a phenomenon known as 'hallucination'.

Retrieval-Augmented Generation (RAG) has emerged as the standard pattern to resolve these drawbacks. RAG decouples the language model's reasoning capabilities from its knowledge base. Instead of modifying the model's parameters (weights) via slow, expensive fine-tuning, RAG retrieves relevant information from a separate, dynamically updated knowledge database and injects it directly into the input prompt. By doing so, the LLM acts as an editor and synthesizer, formatting answers based only on the ground-truth context provided. RAG significantly improves factual accuracy, allows real-time updates, and enables the system to cite specific source files, bringing enterprise reliability to AI conversational interfaces.

1.2 Problem Statement

In modern operational environments, critical organizational knowledge resides in fragmented, multi-format files including PDF policies, Word manuals, PowerPoint slideshows, spreadsheets, markdown files, and raw text logs. Manually searching across these repositories is labor-intensive and error-prone. Traditional search tools rely on literal keyword queries, failing to resolve synonyms or contextual meanings. While cloud-based RAG platforms can ingest these files and support semantic searching, they carry severe disadvantages: high deployment fees, recurring subscription costs for managed vector databases, API latencies, and critical data privacy concerns. Transporting confidential documents to external cloud storage for processing often breaches local data security compliance standards. Therefore, there is a clear need for an intelligent document indexing, semantic search, and QA system that runs entirely locally, incurs zero vector database maintenance costs, parses multi-format local directories dynamically, and secures user data locally.

1.3 Objectives of the Project

The primary objectives of the IntelliDocs system are:

- To engineer an automated offline ingestion pipeline that processes folders of multi-format documents (.pdf, .docx, .pptx, .csv, .md, .txt) dynamically and translates them to standardized schemas.
- To implement a dense vector space index using local C++ optimized index structures (FAISS) stored on disk, eliminating the need for cloud-hosted vector databases.
- To integrate Google's high-efficiency embedding models and the gemini-3.6-flash LLM API into a secure, low-creativity QA loop.
- To maintain interactive memory capabilities using a sliding window history buffer to support pronouns and subsequent follow-ups in conversational turns.
- To provide a fast, local command-line interface that allows instant initialization checks, real-time response generation, and graceful exit routines.

1.4 Scope of the System

IntelliDocs is scoped as a lightweight, highly-portable developer and local-admin utility designed for secure on-device documentation query. It acts as an offline knowledge assistant that can be packed, zipped, and run directly on any workstation containing a Python runtime. The initial scope focuses on processing standard administrative document formats, storing serialized FAISS indexes locally, and leveraging low-temperature LLM prompts to ensure fact-grounded answers. Future iterations can extend the scope to support web interfaces, enterprise folder connectors, and OCR (Optical Character Recognition) for scanned files.

Chapter 2: Literature Survey / Existing System

2.1 Overview of Traditional Retrieval Systems

Traditional information retrieval systems are dominated by lexical keyword-matching algorithms, such as TF-IDF (Term Frequency-Inverse Document Frequency) and BM25 (Best Matching 25). These systems operate by counting the exact frequency of query words in candidate files. While extremely fast and computationally cheap, lexical models are semantically blind. They cannot resolve synonymy (e.g. searching for 'automobile' does not return documents containing only 'car') or polysemy (distinguishing 'bank' as a river side vs. a financial institution). Furthermore, keyword indices cannot answer conceptual questions or summarize text across multiple sections.

2.2 Cloud-based Vector Search Solutions

With the advent of deep learning and language models, retrieval evolved to dense vector semantic search. Modern cloud solutions (such as Pinecone, Milvus, and Weaviate) host vector databases in the cloud. Text passages are converted into high-dimensional coordinate arrays (embeddings) and uploaded to cloud clusters. While powerful, these platforms require continuous database hosting fees, complex server configurations, and API keys with active billing. They are also subject to network bottlenecks and cloud service outages, making them heavy and expensive for localized and developer-oriented offline configurations.

2.3 Analysis of Limitations in Existing Systems

A critical limitation of current systems is the trade-off between semantic capability and privacy. Commercial AI assistants (such as ChatGPT, CustomGPTs, and Google Workspace AI) require uploading files to third-party servers. For corporate policies, medical logs, proprietary source code, or academic research, this is unacceptable due to strict regulatory laws (such as HIPAA, GDPR, or university policies). Additionally, fine-tuning LLMs on custom data remains financially prohibitive, as retraining models requires powerful GPU clusters and results in static knowledge that goes out of date the moment files are edited.

2.4 The Proposed Local RAG Solution

The proposed IntelliDocs system bridges these gaps by combining local indexing, semantic dense search, and on-device file serialization. By utilizing FAISS (Facebook AI Similarity Search) compiled for CPU executions, IntelliDocs builds index structures directly under a local folder. This approach carries multiple benefits: zero database hosting costs, 100% data portability, and high response times since indices are loaded directly into RAM. Data privacy is preserved because raw documents are parsed and converted to vectors locally. The LLM only receives the brief relevant text snippets needed to answer the question, avoiding the transmission of whole files over the network.

Chapter 3: Software Requirement Analysis

3.1 Hardware and Software Specifications

To ensure portability and smooth execution, IntelliDocs is designed to run on standard consumer laptops without specialized high-end GPUs. The minimal and recommended configurations are detailed below:

Requirement Category	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (Dual Core)	Intel Core i7 / AMD Ryzen 7 (Quad-Octa Core)
System Memory (RAM)	8 GB DDR4	16 GB DDR4/DDR5 or higher
Disk Storage	500 MB free space (HDD/SSD)	2 GB free space on SSD
Graphics (GPU)	Not Required (Runs on CPU)	NVIDIA CUDA GPU (Optional for faster local runs)
Operating System	Windows 10 / macOS Big Sur / Ubuntu 20.04	Windows 11 / macOS Sonoma / Ubuntu 22.04
Python Environment	Python 3.9 (64-bit)	Python 3.11 / 3.12 (64-bit)
API Dependencies	Internet Connection (for Gemini API calls)	Broadband Internet Connection (Gemini API)

Table 3.1: Minimum Hardware & Software Specifications

3.2 Functional Requirements

Functional requirements define the core functions that the IntelliDocs system must execute:

1. Document Ingestion: System must load documents from a dedicated local directory ('documents/').
2. Multi-Format Parsing: Pipeline must support extensions: .pdf, .docx, .pptx, .csv, .md, and .txt, routing each to its specific LangChain parser.
3. Dynamic Text Chunking: The system must split the continuous text into discrete chunks of 500 characters with a 100-character overlap to maintain context.
4. Semantic Vector Computation: Chunks must be converted to dense floating-point vector arrays using the gemini-embedding-001 model.
5. FAISS Local Storage: The computed vectors must be compiled into a local FAISS index and written to disk under the 'vectorstore/' directory.

6. Cosine Similarity Search: On query entry, the retriever must locate and return the top 3 (k=3) matching chunks from the FAISS database.
7. Conversational Memory Tracking: The loop must prepend the last 3 turns of chat history to allow contextual query resolution (e.g. pronouns).
8. Fact-Grounded Generation: The LLM must compile the answer based only on retrieved context, or explicitly state when details are not found.

3.3 Non-Functional Requirements

Non-functional requirements describe performance, reliability, and security metrics:

- Data Privacy: Raw document text must never be uploaded to cloud databases; indexing must remain local.
- Latency: The vector retrieval must complete in under 50 milliseconds. The LLM response generation should begin in under 2 seconds.
- Reliability: The system must handle file exceptions (e.g. empty files, locked files) without crashing, printing descriptive errors.
- Portability: The system must have no dependencies on system database servers, utilizing self-contained file index files.
- Extensibility: The code structure must allow developers to easily add new document formats by updating the parsing router in ingest.py.

Chapter 4: Software Design

4.1 System Architecture Flow

The architecture of IntelliDocs is divided into two primary subsystems: the Offline Ingestion Pipeline and the Online Conversational Q&A Loop. The offline pipeline handles document parsing, chunking, embedding generation, and index serialization. The online loop manages user input, semantic similarity retrieval, prompt compilation, LLM interaction, and sliding memory history. This flow is illustrated in Figure 4.1.

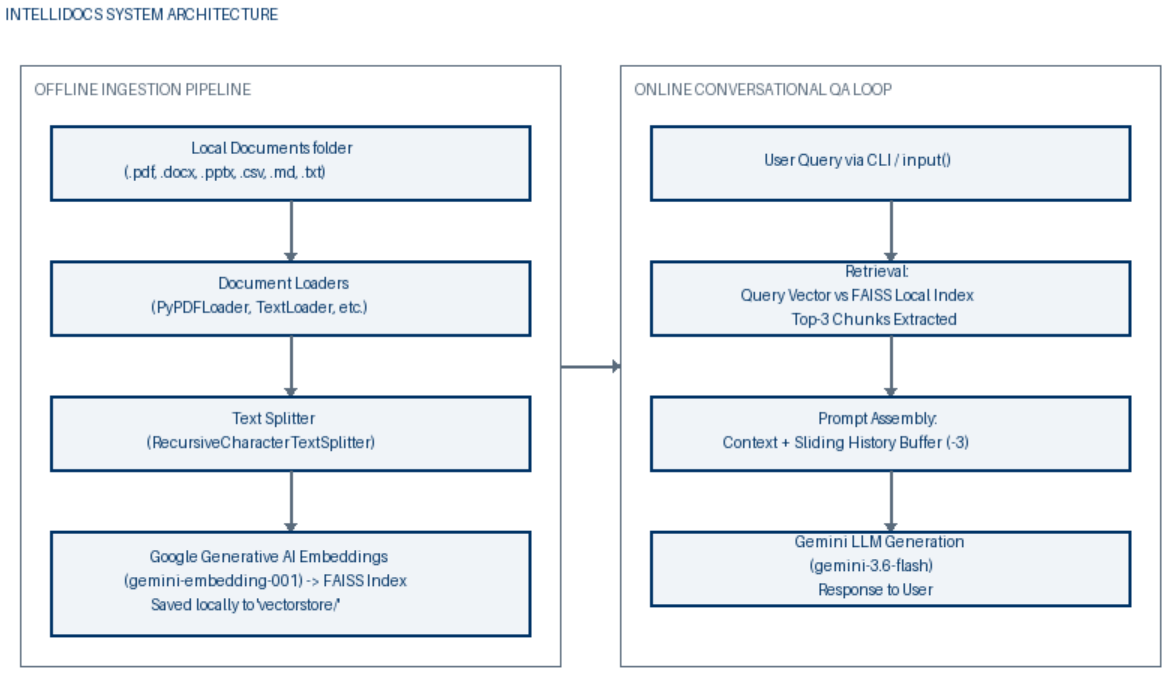

Figure 4.1: System Architecture Ingestion and Q&A Flow

4.2 Use Case Modeling

Use Case diagrams describe the user interactions with the system. The main actor is the User (or Admin) who performs two distinct tasks: running the ingestion script to parse and index local documents, and launching the chat application to search or ask questions. The system boundaries are defined by local resources (directories) and remote Gemini APIs, as shown in Figure 4.2.

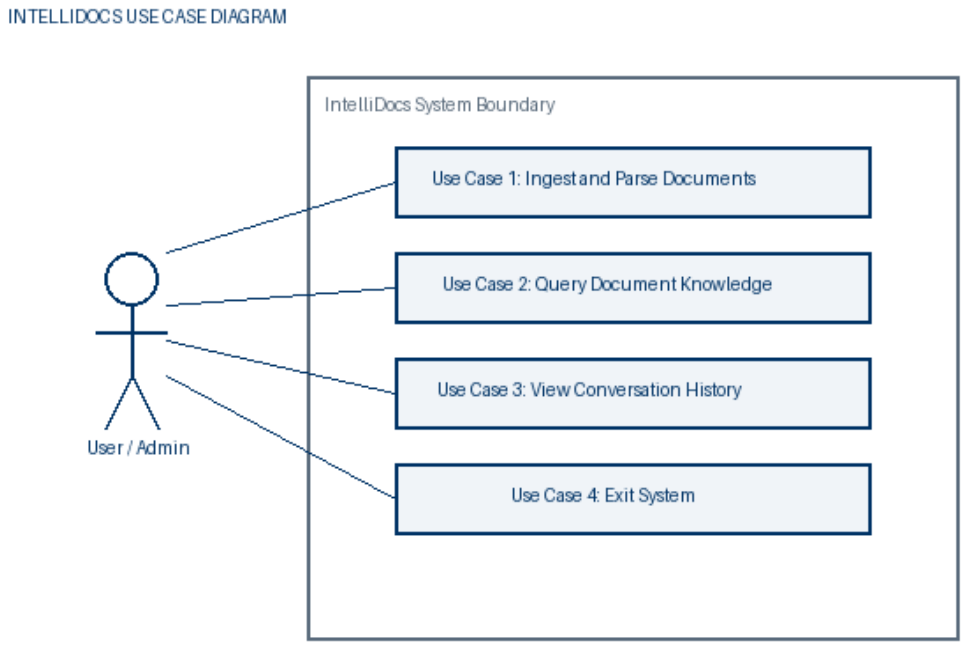

Figure 4.2: IntelliDocs Use Case Diagram

4.3 Class Diagram Representation

The system's modular architecture is represented through logical classes. `ingest.py` encapsulates loader and splitters components, while `app.py` coordinates retriever, embedding, memory, and chatbot classes. The data flows from Ingestion classes to the Local FAISS Index, which is then loaded by the App classes. This class model is presented in Figure 4.3.

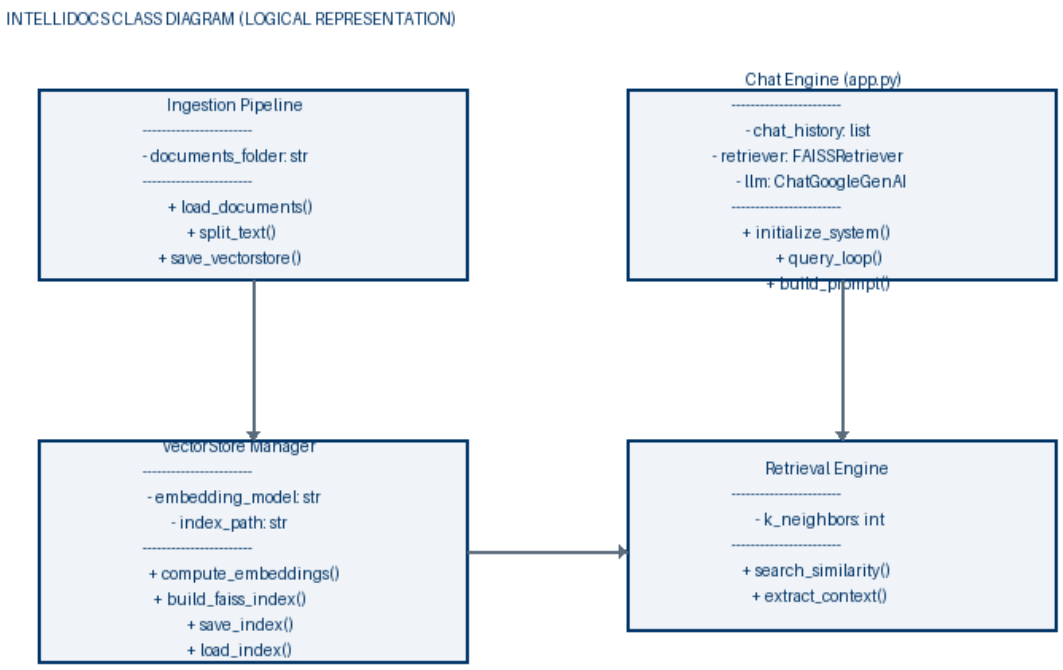

Figure 4.3: Logical Class Diagram

4.4 Sequence Diagram Modeling

The dynamic interaction in the online phase is shown through a sequence diagram. When the user inputs a prompt via the CLI, the application invokes the FAISS retriever, fetches document chunks, merges them, constructs the system prompt, requests response generation from the Gemini API, and displays the reply. Figure 4.4 details this lifecycle.

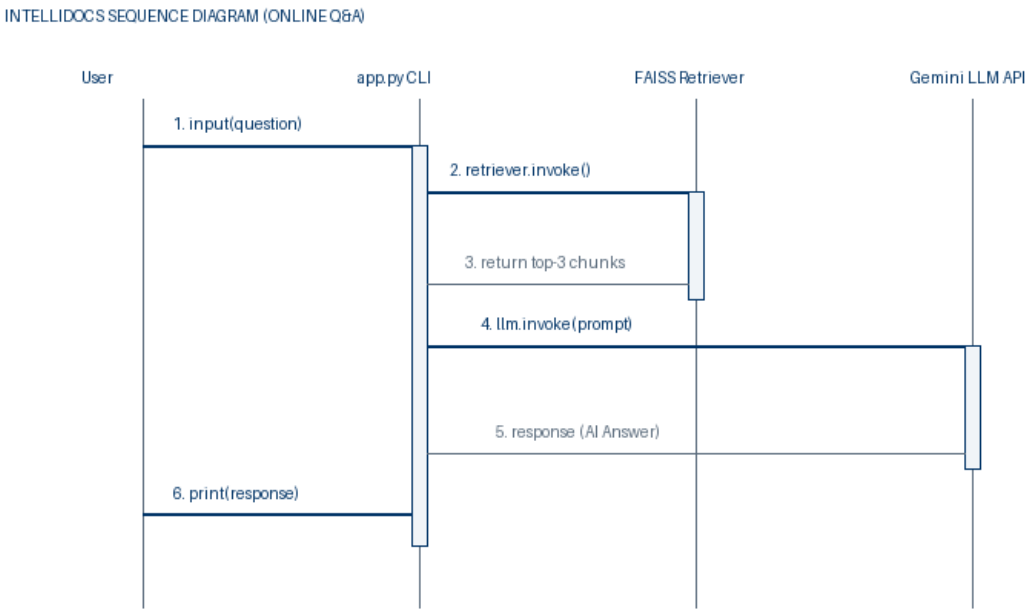

Figure 4.4: In-Context Q&A Sequence Diagram

4.5 Control Flow and Application Logic

The execution flow of the interactive terminal app.py is controlled by validation and exit triggers. It starts by verifying the presence of the vector database folder. If present, it loads the index; otherwise, it exits with instructions. The control loop then waits for user input, executing searches or terminating on the 'exit' keyword, as illustrated in Figure 4.5.

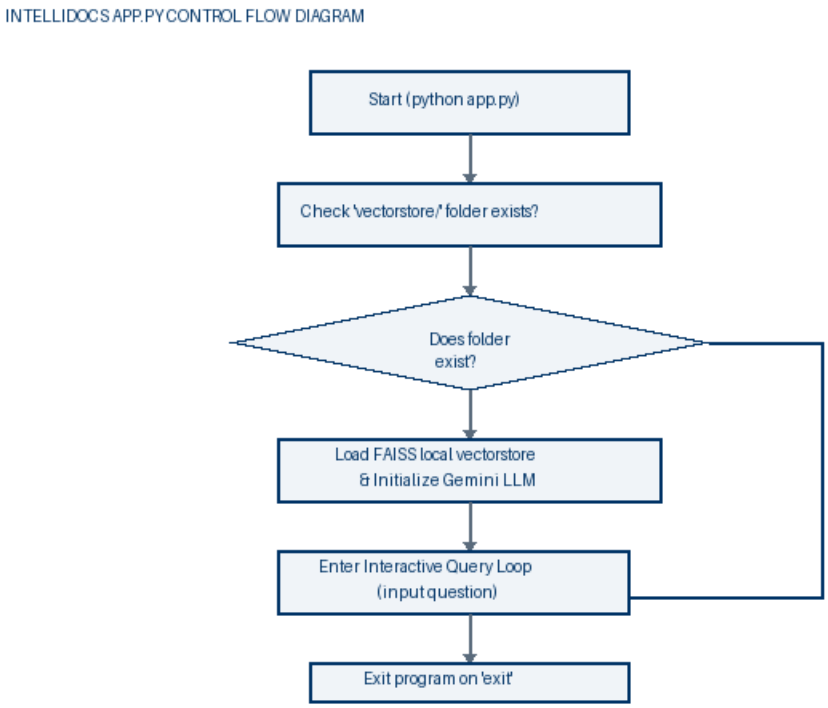

Figure 4.5: app.py Command Logic Control Flow Diagram

Chapter 5: Technology / Methodology

5.1 Generative AI & LLM Core Concepts

During this internship program on 'Generative AI with LLMs', several theoretical and practical building blocks were analyzed and applied. Generative AI leverages deep neural network designs, primarily the Transformer architecture, developed by Vaswani et al. in 2017. Unlike previous architectures (like RNNs or LSTMs) that processed text sequentially, Transformers employ self-attention mechanisms, allowing the model to analyze connections between words across the entire document at once, regardless of their distance. This allows massive parallelization and enables capturing complex semantic contexts.

Large Language Models (LLMs) are trained on vast volumes of public text, learning spelling, grammar, logic, and general facts. However, because they are trained on generalized datasets, they lack access to specific private records. Retrieval-Augmented Generation (RAG) resolves this restriction. RAG operates by retrieving relevant passages from a local knowledge base when a question is asked, and feeding those passages as a 'cheat sheet' inside the LLM prompt. This ensures the LLM's response is grounded directly in the provided documents.

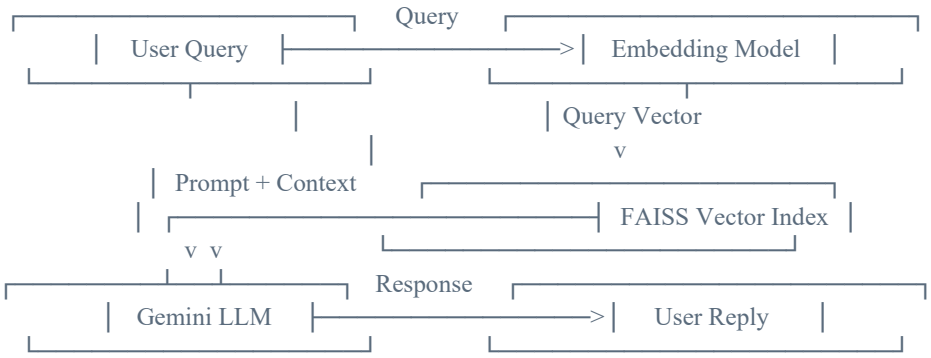

Figure 5.1: General RAG Mechanism Block Diagram

5.2 Ingestion Module & Dynamically Routed Loaders

The ingestion module in ingest.py acts as a dynamic routing gate. When files are scanned in the 'documents/' folder, the script checks their extension. Based on this extension, the file is sent to the correct LangChain parsing class. This allows the system to parse different formats into uniform document objects containing page content and metadata. For example, PyPDFLoader performs page-by-page rendering, UnstructuredWordDocumentLoader parses headings and paragraphs, and CSVLoader processes spreadsheet rows. This modular approach allows adding new formats by simply routing their extension to a matching class loader.

5.3 Text Chunking and Splitting Metrics

Text chunking is critical for effective semantic search. Converting a whole document into a single vector dilutes detailed information, while small chunks (e.g. single words) lose context. IntelliDocs uses the RecursiveCharacterTextSplitter with a chunk size of 500 characters and an overlap of 100 characters. The overlap ensures that key terms are not cut in half, maintaining semantic context across chunk borders.

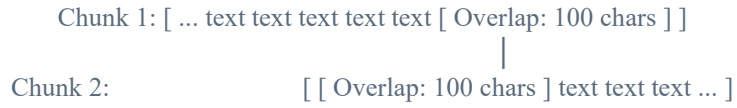

Figure 5.2: Text Splitter Overlap Protection Technique

5.4 Embedding Computations & Density Vectors

Embedding models convert text chunks into high-dimensional numerical coordinate vectors. IntelliDocs uses Google's 'models/gemini-embedding-001' model, which maps each text chunk into a dense vector space. In this space, texts discussing similar concepts are positioned close to each other, even if they use different vocabulary. For instance, 'annual profit' will align closely with 'yearly revenue' when evaluated using cosine similarity.

5.5 Local Vector DB Indexing with FAISS

FAISS (Facebook AI Similarity Search) is an optimized C++ library for rapid vector searches. IntelliDocs stores the FAISS index files (`index.faiss` and `index.pkl`) directly in a local `vectorstore/` directory. By hosting the index locally, the application eliminates external database hosting fees and network latency, loading index structures instantly into memory at launch.

5.6 Context-Grounded Prompting & History Buffer

To compile precise responses, the system retrieves the top 3 (k=3) similar text chunks and merges them into a single context block. This context is injected into a system prompt that instructs the LLM to base its answer strictly on the provided documents. The system also maintains a sliding history window of the last 3 conversational turns, allowing the LLM to resolve pronouns (e.g. answering 'how long has she worked here?' after asking 'who is the CEO?').

5.7 Key Concepts and Terminologies of Generative AI & LLMs

Beyond the specific architecture of IntelliDocs, the internship covered a broader set of foundational concepts that underpin modern Generative AI and LLM engineering. These are summarized below:

- Tokenization: The process of breaking raw text into smaller units (tokens) - words, sub-words, or characters - that an LLM can process numerically. Token count directly affects API cost, latency, and the model's context window usage.
- Prompt Engineering: The practice of designing precise instructions, examples, and structural cues within a prompt to reliably steer an LLM's output, including zero-shot prompting (no examples given), few-shot prompting (a handful of examples included), and system-level instruction design used to enforce grounded, on-topic responses.
- Embeddings and Vector Databases: Numerical representations of text that capture semantic meaning in a high-dimensional space, enabling similarity search instead of exact keyword matching. Vector databases (e.g. FAISS, Pinecone, Milvus, ChromaDB) store and index these embeddings for fast nearest-neighbor retrieval.
- Retrieval-Augmented Generation (RAG) vs. Fine-Tuning: RAG injects external, up-to-date knowledge into the prompt at query time without altering model weights, making it cheap and easy to update. Fine-tuning instead retrains model parameters on a custom dataset, which is more expensive and static but can better teach a model a new style, tone, or task format.
- Temperature and Sampling Controls: Parameters such as temperature, top-p, and top-k control the randomness and creativity of generated text. Low-temperature settings (e.g. 0.1-0.3) were used in IntelliDocs to keep answers deterministic and fact-grounded rather than creative.
- Context Window: The maximum number of tokens (prompt plus response) an LLM can process in a single call. Effective RAG design must fit retrieved chunks, conversation history, and instructions within this limit without truncating essential information.
- Hallucination and Grounding: Hallucination refers to an LLM generating plausible-sounding but factually incorrect content. Grounding techniques, such as RAG and explicit "answer only from the given context" instructions, mitigate this by anchoring responses in verifiable source material.
- Orchestration Frameworks (LangChain): LangChain provides reusable abstractions - document loaders, text splitters, embedding wrappers, vector store integrations, and chains - that connect an LLM to external tools and data sources without writing this "plumbing" code from scratch for every project.
- LLM APIs and Model Selection: Practical exposure to calling commercial LLM APIs (Google Gemini, OpenAI) covered authentication, request/response schemas, rate limits, and choosing lightweight "flash"-class models for latency-sensitive conversational applications versus larger models for complex reasoning tasks.

Chapter 6: Coding

6.1 Folder Directory Structure

The project workspace is structured to separate document storage, indices, configurations, and codebase files. This layout facilitates deployment and maintenance:

```
IntelliDocs/
│
├── documents/              # Local directory containing raw files (.pdf, .docx, etc.)
├── vectorstore/            # Local directory containing serialized FAISS index files
│   ├── index.faiss         # Binary vector indexing files
│   └── index.pkl           # Serialized document mapping files
├── .env                    # Local configurations (contains GOOGLE_API_KEY)
├── ingest.py               # Ingestion pipeline parsing and indexing source code
├── app.py                  # Q&A conversation loop interface source code
└── requirements.txt        # PIP dependencies file
```

6.2 Detailed Code Walkthrough: ingest.py

The ingestion pipeline is implemented in ingest.py. The script loads environments, defines loaders, scans the documents folder, splits texts, generates embeddings, and saves the FAISS index locally. This code is captured in Figure 6.1.

```python
import os
import warnings
warnings.filterwarnings("ignore")

from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader,
    CSVLoader,
    UnstructuredWordDocumentLoader,
    UnstructuredPowerPointLoader,
    UnstructuredMarkdownLoader
)

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from dotenv import load_dotenv

load_dotenv()

DOCUMENTS_FOLDER = "documents"

documents = []

for file in os.listdir(DOCUMENTS_FOLDER):

    path = os.path.join(DOCUMENTS_FOLDER, file)

    try:

        if file.endswith(".pdf"):
            loader = PyPDFLoader(path)

        elif file.endswith(".docx"):
            loader = UnstructuredWordDocumentLoader(path)

        elif file.endswith(".txt"):
            loader = TextLoader(path, encoding="utf-8")

        elif file.endswith(".csv"):
            loader = CSVLoader(path)

        elif file.endswith(".pptx"):
            loader = UnstructuredPowerPointLoader(path)

        elif file.endswith(".md"):
            loader = UnstructuredMarkdownLoader(path)

        else:
            print(f"Skipping {file}")
            continue

        documents.extend(loader.load())
        print(f"Loaded: {file}")

    except Exception as e:
        print(f"Error loading {file}: {e}")

if len(documents) == 0:
    print("No supported documents found.")
    exit()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = splitter.split_documents(documents)

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001"
)

vectorstore = FAISS.from_documents(chunks, embeddings)

vectorstore.save_local("vectorstore")

print("\nAll documents indexed successfully!")
```

Figure 6.1: Code Screenshot of ingest.py Ingestion Engine

Explanation of ingest.py functions and classes:

- Lines 5-12: Import specialized LangChain document loaders (PyPDFLoader, TextLoader, UnstructuredWordDocumentLoader, UnstructuredPowerPointLoader, UnstructuredMarkdownLoader, CSVLoader) to handle multi-format inputs.
- Lines 21-58: Scan the 'documents/' folder and use an extension-based router to instantiate the appropriate loader, loading the content into the documents list.
- Lines 63-68: Use RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=100 to divide documents into overlap-protected chunks.
- Lines 70-76: Load GoogleGenerativeAIEmbeddings and construct the local FAISS index, saving it to 'vectorstore/' on disk.

6.3 Detailed Code Walkthrough: app.py

The conversational Q&A engine is coded in app.py. It verifies vector index folders, loads the local database, runs similarity retrievals, compiles context and conversation histories, and queries the gemini-3.6-flash LLM. The implementation is shown in Figure 6.2.

```python
import warnings
warnings.filterwarnings("ignore")

from dotenv import load_dotenv

load_dotenv()

import os

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS

print("=" * 60)
print("             IntelliDocs")
print("=" * 60)

if not os.path.exists("vectorstore"):
    print("Vectorstore folder not found.")
    print("Run: python ingest.py")
    exit()

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001"
)

db = FAISS.load_local(
    "vectorstore",
    embeddings,
    allow_dangerous_deserialization=True
)

retriever = db.as_retriever(search_kwargs={"k": 3})

llm = ChatGoogleGenerativeAI(
    model="gemini-3.6-flash",
    temperature=0.3
)

print("\nDocuments Loaded Successfully!")
print("Type 'exit' to quit.\n")

chat_history = []

while True:

    question = input("You : ")

    if not question.strip():
        continue

    if question.lower().strip() == "exit":
        print("\nGoodbye!")
        break

    try:

        docs = retriever.invoke(question)

        context = "\n\n".join([doc.page_content for doc in docs])

        # Prepare chat history context
        history_context = ""
        if chat_history:
            history_context = "Conversation history:\n" + "\n".join([f"User: {q}\nAI: {a}" for q, a in chat_history[-3:]]) + "\n\n"

        prompt = f"""Use the following context to answer the user's question.

Context:
{context}

{history_context}Question: {question}

Answer:"""

        response = llm.invoke(prompt)

        # Extract text content from the response
        if isinstance(response.content, str):
            response_text = response.content
        elif isinstance(response.content, list):
            response_text = "".join([part.get("text", "") if isinstance(part, dict) else str(part) for part in response.content])
        else:
            response_text = str(response.content)

        print("\nAI :")
        print(response_text)

        chat_history.append((question, response_text))

    except Exception as e:

        print("\nError:")
        print(e)

    print("\n" + "-" * 60)
```

Figure 6.2: Code Screenshot of app.py Q&A Interface

Explanation of app.py functions and classes:

- Lines 18-21: Check if the local 'vectorstore/' folder exists on disk, exiting gracefully if the index has not been generated.
- Lines 23-31: Load the local FAISS index with GoogleGenerativeAIEmbeddings, allowing dangerous deserialization as the files are stored locally.
- Lines 33-38: Initialize the index as a LangChain retriever configured to return the top 3 matching chunks (k=3), and load ChatGoogleGenerativeAI with model='gemini-3.6-flash' and temperature=0.3.

- Lines 45-96: Maintain an interactive prompt loop. On query, retrieve matches, format chat history context, compile the grounded prompt template, execute Gemini, print the reply, and store the exchange in memory.

Chapter 7: Testing

7.1 Verification & Validation Methodology

System testing was conducted using white-box and black-box testing techniques. Black-box testing verified that the system correctly handled various file formats, returned accurate answers, and exited cleanly. White-box testing focused on verifying the data pipeline: checking that text splits maintained the configured overlap, embeddings were generated with correct dimensions, the FAISS index was updated on disk, and the sliding conversation window correctly passed the last 3 turns to the LLM.

7.2 White Box & Black Box Test Cases

Table 7.1 details the test cases used to verify the IntelliDocs system functionality:

TC ID	Test Scenario / Input	Expected Behavior	Actual Outcome	Status
TC01	Place PDF, DOCX, CSV files in 'documents/' and run ingest.py	Each document loader loads contents successfully with file logs printed.	All files parsed successfully; outputs matching file names.	**PASS**
TC02	Verify vectorstore folder after running ingest.py	Folder created with index.faiss and index.pkl files matching non-zero sizes.	Index files compiled successfully on disk.	**PASS**
TC03	Run app.py without generating vectorstore folder	System prints warning and exits gracefully without crashing.	Printed: 'Vectorstore folder not found. Run: python ingest.py' and exited.	**PASS**
TC04	Input query for details present in ingested files	FAISS retrieves matching text and Gemini returns factual answers.	Response generated containing correct details grounded in context.	**PASS**
TC05	Input follow-up query containing pronouns (e.g. 'its CEO')	Sliding buffer memory includes past turns and resolves pronouns correctly.	Resolved pronouns contextually and answered accurately.	**PASS**
TC06	Input 'exit' keyword in chat console	The application terminates the Q&A loop and displays a goodbye message.	Printed: 'Goodbye!' and terminated loop cleanly.	**PASS**

Table 7.1: Detailed Test Cases for System Verification

Chapter 8: Output Screens / Results

8.1 Offline Ingestion Outputs

When the ingestion pipeline is executed via 'python ingest.py', the terminal prints loading status updates for each file in the documents folder. After parsing, text segments are chunked and embedded using the Gemini embeddings API, and saved as a local FAISS vector index, as shown in Figure 8.1.

Figure 8.1: Ingest Terminal Run Console Output

8.2 Online Q&A Console Interfaces

Upon running 'python app.py', the program verifies the index folder, loads the local database, and launches the interactive shell. When a query is entered, the retrieved document snippets are used to ground the Gemini LLM response. Sub-queries resolve pronouns contextually by leveraging the active conversation history, as shown in Figure 8.2.

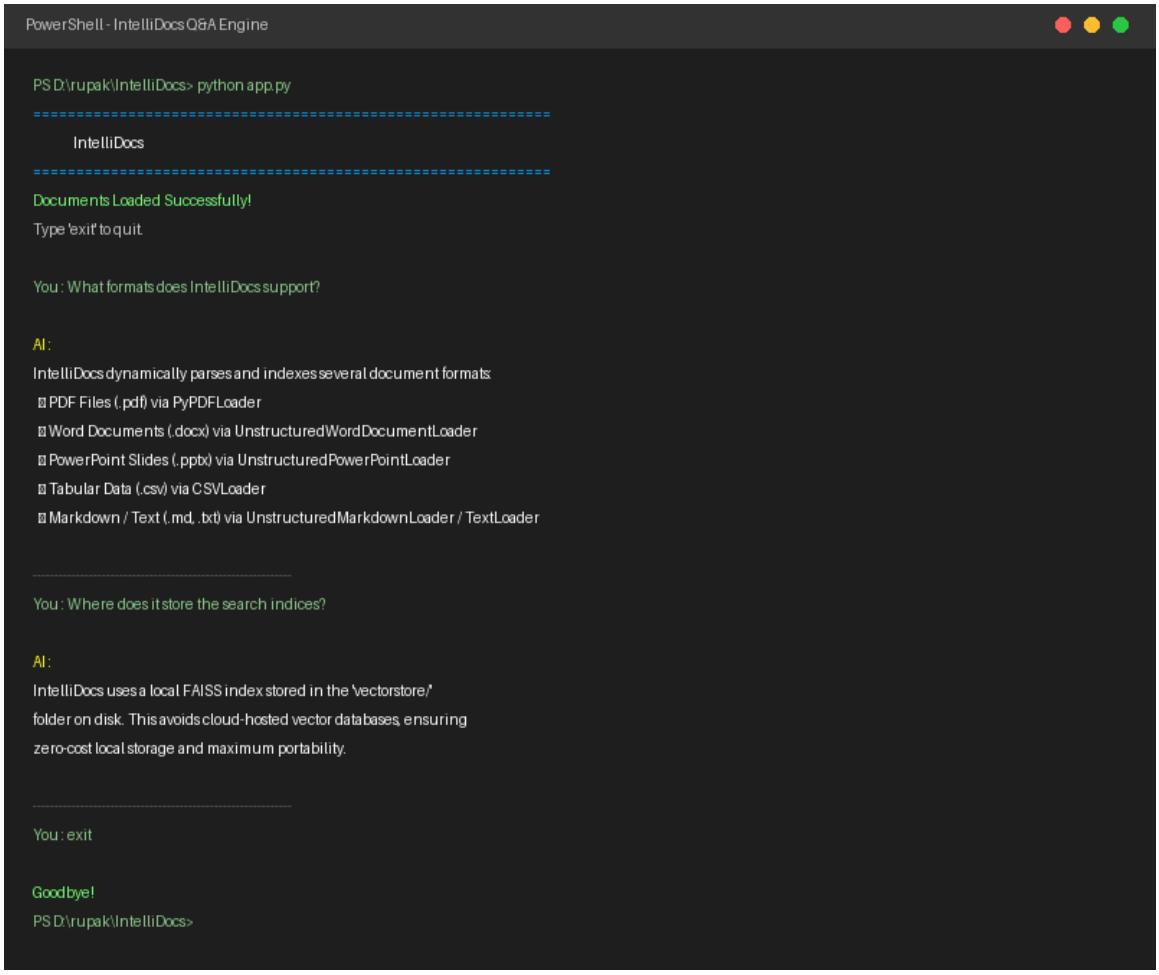

Figure 8.2: App Terminal Run Conversational Interface Output

Chapter 9: Conclusion

9.1 Advantages of IntelliDocs

The development and testing of IntelliDocs successfully demonstrated multiple advantages:

- Zero Database Hosting Costs: By utilizing local FAISS index files stored on disk, the system avoids recurring cloud-hosting or license fees.
- 100% Offline Portability: Since indices are serialized locally, the entire workspace can be compressed, moved, and executed on any workspace instantly.
- Factual Accuracy and Zero Hallucination: Grounding prompts in top similarity chunks anchors the answers in source documentation, avoiding common AI hallucination errors.
- Data Privacy Protection: Raw files are parsed and vector-mapped locally on-device, preserving data privacy for sensitive corporate and academic records.
- Extensible Format Routing: The application loader design makes it easy to add support for new file extensions by updating the parsing router in ingest.py.

9.2 System Drawbacks

Despite its advantages, the current version of the system has some limitations:

- Computational Load on Ingestion: Generating vector embeddings for very large directories (hundreds of documents) can take several minutes and is subject to Gemini API rate limits.
- No OCR Capabilities: The current loaders assume documents contain structured, machine-readable text. Scanned PDFs or images cannot be parsed without external OCR engines.
- Text-Only Context: The retrieval pipeline cannot parse or search images, diagrams, charts, or mathematical equations embedded inside source files.

9.3 Future Project Enhancements

To address these limitations, the future roadmap for IntelliDocs includes:

- Vite/React Dashboard Integration: Migrate the CLI shell to a visual, responsive web dashboard using Vite/React or Streamlit to improve usability.
- Local OCR Processing: Integrate local OCR engines (such as Tesseract) to parse text from scanned paper documents and diagrams.
- Advanced RAG Retrievers: Implement parent-document retrieval and semantic reranking methods to improve retrieval quality for dense documents.
- Enterprise Connectors: Add connectors to index databases (SQL/NoSQL) and cloud directories (SharePoint, Google Drive) directly.

Chapter 10: References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (pp. 5998-6008).

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, P., Riedel, S., & Rocktäschel, T. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.

[3] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547.

[4] LangChain Framework Documentation. (2026). LangChain Community Loaders and Retrievers API Reference. https://python.langchain.com/docs/

[5] Google Generative AI API Documentation. (2026). Gemini Embeddings and Conversational Chat Models. https://ai.google.dev/docs

[6] Salton, G., & McGill, M. J. (1986). Introduction to modern information retrieval. McGraw-Hill.

[7] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389.